



Итоговый проект. Применение ИИ и AI-агентов в реальной рабочей задаче

Задание

Выберите одну реальную задачу из своей текущей работы, рабочего проекта, внутреннего проекта компании или регулярно выполняемого процесса и попробуйте улучшить её с помощью ИИ.

Не требуется придумывать отдельный учебный проект специально для курса. Предпочтительный вариант — взять задачу, с которой вы уже сталкиваетесь на текущем месте работы и понимаете её контекст.

Задача может быть небольшой. Цель итоговой работы — не создать сложную мультиагентную платформу, а показать, что вы умеете определить подходящий сценарий применения ИИ, спланировать решение, реализовать его в разумном объёме и оценить результат.

Допускаются задачи, связанные с разработкой, тестированием, сопровождением, документацией, анализом кода, обработкой логов, Git, CI/CD и другими процессами разработки ПО.

Примеры подходящих рабочих задач:

- автоматизация проверки изменений перед отправкой Pull Request / Merge Request;
 - генерация unit-тестов для существующего модуля;
 - генерация integration-тестов;
 - анализ логов и подготовка гипотез о причинах ошибки;
 - автоматизация первичного code review;
 - поиск потенциально проблемных участков кода;
 - рефакторинг существующего модуля;
 - обновление устаревших методов или зависимостей;
 - подготовка документации для существующего проекта;
 - генерация или обновление README;
 - анализ изменений между версиями;
 - поиск мест использования устаревшего API;
 - подготовка плана миграции;
 - автоматизация части работы с Git;
 - анализ Pull Request;
 - проверка проекта линтером с последующим анализом ошибок через ИИ;
 - подготовка тестовых сценариев по требованиям;
 - анализ баг-репортов;
 - генерация гипотез по дефектам;
 - автоматизация повторяющейся операции разработчика или тестировщика;
 - создание простого AI-агента для работы с проектом;
-

- создание нескольких агентов с различными ролями, если это действительно оправдано задачей.

Можно выбрать другую задачу, если она относится к вашей реальной работе и позволяет продемонстрировать применение материала курса.

Цель задания

Разработать небольшое практическое AI-решение для реальной рабочей задачи и показать полный путь от постановки проблемы до проверки результата.

В итоговой работе необходимо продемонстрировать понимание следующих принципов:

- выбор задачи, в которой применение ИИ имеет практический смысл;
- определение границ автоматизации;
- выбор подходящего AI-инструмента или модели;
- определение роли человека и роли ИИ;
- постановка инструкций для модели или агента;
- работа с существующим кодом, документацией, логами или другими данными проекта;
- использование инструментов проекта;
- проверка результата;
- контроль ошибок и рисков;
- документирование решения.

Не обязательно использовать все технологии, рассмотренные в курсе.

Например, если для вашей задачи достаточно одного AI-агента, не нужно искусственно создавать три или четыре агента. Если задача не требует RAG, его не нужно добавлять только для демонстрации технологии. Если интеграция с CI не имеет смысла для выбранной задачи, достаточно показать локальный запуск и объяснить возможную точку интеграции.

Оценивается не количество использованных технологий, а обоснованность выбранного решения.

Задание: шаги выполнения

Шаг 1. Выберите реальную рабочую задачу

Выберите процесс или проблему, с которой вы действительно сталкиваетесь в работе.

Кратко опишите исходную ситуацию.

Нужно указать:

- что сейчас выполняется;
- кто выполняет эту работу;
- какие данные используются;
- какой результат должен быть получен;
- какая часть работы занимает время или регулярно повторяется;
- какую часть потенциально можно передать ИИ.

Не выбирайте слишком большую задачу.

Например, задача «автоматизировать всю разработку проекта» слишком широкая.

Задача «анализировать изменения в Pull Request и формировать предварительные замечания перед ручным review» подходит значительно лучше.

Шаг 2. Опишите проблему

Объясните, что именно требуется улучшить.

Например:

Сейчас разработчик вручную просматривает логи приложения после неудачного запуска. Логи могут содержать несколько сотен строк. Требуется создать инструмент, который получает лог, выделяет наиболее важные ошибки и формирует несколько возможных причин проблемы.

Или:

В проекте регулярно появляются Pull Request с одинаковыми типовыми ошибками. Требуется создать AI-ревьюера, который анализирует git diff и формирует предварительные замечания до ручного review.

Описание должно быть связано именно с вашей задачей.

Шаг 3. Определите ожидаемый результат

До начала реализации определите, что будет считаться успешным результатом.

Например:

- агент корректно определяет изменённые файлы;
 - анализирует только изменения, а не весь репозиторий;
 - обнаруживает потенциальные ошибки;
-

- не изменяет файлы без разрешения;
- создаёт предложения по исправлению;
- формирует отчёт;
- запускает тесты после изменений;
- создаёт документацию;
- правильно обрабатывает ошибочный вход;
- сокращает количество ручных действий.

Необходимо выбрать несколько критериев, которые реально можно проверить.

Не используйте формулировку:

«ИИ должен хорошо работать».

Лучше определить конкретное поведение:

«Из пяти заранее подготовленных примеров агент должен правильно определить минимум четыре ошибки».

Шаг 4. Определите роль ИИ

Опишите, какую именно работу будет выполнять ИИ.

Можно использовать один из подходов:

- ИИ только предлагает решение, человек принимает окончательное решение;
- ИИ выполняет часть действий, человек проверяет результат;
- агент автоматически выполняет ограниченный набор действий;
- несколько агентов выполняют разные этапы работы.

Укажите, где обязательно должен оставаться человеческий контроль.

Например:

Агент может анализировать код и предлагать изменения, но не имеет права автоматически выполнять merge в основную ветку.

Шаг 5. Выберите архитектуру решения

Выберите минимально достаточную архитектуру.

Это может быть:

- обычный запрос к LLM;
- AI-ассистент в IDE;
- один агент;
- агент с подключёнными инструментами;
- агент с доступом к Git;
- агент с доступом к документации проекта;
- агент с RAG;
- несколько агентов;
- AI-проверка внутри CI/CD.

Использование мультиагентной системы желательно только в тех случаях, где разделение ролей действительно помогает решению задачи.

Например:

Planner — составляет план изменения.

Coder — выполняет изменение.

Reviewer — проверяет результат.

Для маленькой задачи аналогичную работу может выполнять один агент.

Шаг 6. Если используются агенты — определите их роли

Для каждого агента укажите:

- название;
- цель;
- входные данные;
- разрешённые действия;
- используемые инструменты;
- ограничения;
- ожидаемый результат.

Например:

Reviewer

Цель: проверить изменения в Pull Request.

Получает:

- `git diff`;

- изменённые файлы;
- правила проекта.

Может:

- читать код;
- запускать линтер;
- анализировать тесты;
- создавать текстовый отчёт.

Не может:

- выполнять merge;
- удалять файлы;
- изменять production-конфигурацию;
- получать секреты проекта.

Шаг 7. Подготовьте инструкции для ИИ

Создайте системную инструкцию, prompt или конфигурацию агента.

В инструкции желательно определить:

- роль;
- задачу;
- контекст;
- ожидаемый формат результата;
- ограничения;
- критерии проверки;
- действия при недостатке информации.

Например, агент не должен придумывать отсутствующий код или делать выводы о файлах, которые ему не были переданы.

Если в проекте используется `agents.md`, можно оформить роли и ограничения в нём.

Использование именно `agents.md` не является обязательным требованием итоговой работы.

Шаг 8. Подключите необходимые инструменты

Подключайте только те инструменты, которые нужны выбранной задаче.

Например:

- Git;
- `git diff`;
- `git log`;
- поиск по файлам;
- linter;
- formatter;
- unit-тесты;
- integration-тесты;
- документацию проекта;
- API;
- CI/CD;
- базу знаний;
- локальные скрипты.

Если задача решается без дополнительных инструментов, искусственно подключать их не нужно.

Шаг 9. Реализуйте рабочий прототип

Создайте минимально рабочую версию решения.

Прототип должен выполнять заявленную основную функцию.

Например, если выбрана задача AI-review, минимальный прототип должен:

1. получить изменения;
2. передать необходимые данные модели;
3. получить результат анализа;
4. вывести замечания;
5. позволить человеку проверить результат.

Не требуется превращать итоговое задание в полноценный production-продукт.

Шаг 10. Используйте Git

Проект должен находиться под контролем версий.

Желательно создать отдельную ветку для итоговой работы.

Например:

```
git checkout -b ai-final-project
```

В процессе разработки делайте осмысленные коммиты.

Например:

```
git add .  
git commit -m "Добавлена базовая логика AI-ревью"
```

```
git add .  
git commit -m "Добавлен запуск тестов после изменений"
```

```
git add .  
git commit -m "Добавлена обработка ошибок агента"
```

Не требуется создавать большое количество коммитов. История должна показывать основные этапы реализации.

Шаг 11. При возможности добавьте автоматическую проверку

Если выбранная рабочая задача связана с кодом, желательно добавить хотя бы один механизм автоматической проверки.

Например:

- unit-тесты;
- integration-тесты;
- linter;
- formatter;
- статический анализ;
- проверку запуска;
- проверку структуры результата агента.

Например:

```
pytest
```

или:

```
npm test
```

или другой инструмент, используемый в вашем рабочем проекте.

Если автоматические тесты неприменимы к выбранной задаче, опишите другой способ проверки результата.

Шаг 12. При возможности интегрируйте решение с CI

Для задач, где это имеет практический смысл, можно добавить запуск через CI.

Например:

- запуск AI-review при создании Merge Request;
- запуск тестов после изменений;
- проверка документации;
- запуск линтера;
- анализ результатов тестирования.

Полная CI/CD-инфраструктура для итоговой работы не требуется.

Если CI уже используется на рабочем проекте, достаточно показать, в какую часть существующего процесса было добавлено или могло бы быть добавлено ваше решение.

Шаг 13. Проверьте решение на нескольких примерах

Не ограничивайтесь одним успешным запуском.

Проверьте минимум несколько ситуаций.

Например:

Обычная ситуация

Корректные входные данные, ожидаемый результат.

Пограничная ситуация

Очень маленький или большой входной файл, отсутствие изменений, неполные данные.

Ошибка

Недоступный файл, неправильный формат, ошибка инструмента, ошибка теста или некорректный ответ модели.

Зафиксируйте результаты проверки.

Шаг 14. Сравните работу до и после применения ИИ

Опишите, что изменилось.

Например:

До внедрения:

Разработчик вручную просматривал весь `git diff` и проверял типовые ошибки.

После внедрения:

Агент выполняет предварительный анализ изменений и формирует список потенциальных проблем. Разработчик выполняет окончательное review.

Можно сравнить:

- время выполнения;
- количество ручных действий;
- количество обнаруженных ошибок;
- качество документации;
- скорость анализа;
- количество повторяющихся операций.

Точные метрики приветствуются, но не являются обязательными, если их невозможно корректно получить в рамках итогового задания.

Шаг 15. Опишите ограничения

Обязательно укажите, что решение пока не умеет или где ему нельзя полностью доверять.

Например:

- результат модели необходимо проверять человеку;
- возможны ложные замечания;
- агент не получает доступ к production;
- агент работает только с определёнными типами файлов;
- большие репозитории требуют дополнительного управления контекстом;
- стоимость запросов пока не оптимизирована;
- CI-интеграция протестирована только на тестовой ветке.

Описание ограничений считается частью качественной итоговой работы.

Подсказки по ключевым частям

Не усложняйте задачу специально

Итоговая работа не должна превращаться в демонстрацию максимального количества терминов курса.

Если задачу можно решить одним агентом, используйте одного агента.

Если достаточно IDE-ассистента и небольшого скрипта, это допустимый вариант.

Если RAG не нужен, не добавляйте RAG.

Если не требуется отдельный Planner, не создавайте Planner только потому, что он был рассмотрен на занятиях.

Главный вопрос:

Помогает ли этот компонент решить выбранную рабочую задачу?

Берите задачу со своей текущей работы

Приоритет имеют собственные задачи.

Например, если на работе регулярно приходится:

- проверять однотипные изменения;
- анализировать логи;
- писать тесты;
- обновлять документацию;
- переносить код;
- выполнять рефакторинг;
- проверять конфигурации;
- разбирать баг-репорты;
- составлять тест-кейсы;
- просматривать Pull Request;

лучше автоматизировать один из этих процессов, чем создавать искусственный учебный пример.

Ограничьте размер задачи

Хорошая итоговая работа должна быть завершённой.

Лучше полностью автоматизировать один небольшой процесс, чем начать создание большой системы и не довести её до рабочего состояния.

Например, вместо:

```
«Создать AI-систему автоматической разработки проекта»
```

лучше:

```
«Создать агента, который анализирует изменения в Python-модуле, запускает тесты и формирует предварительное review».
```

Сначала определите вход и выход

До реализации зафиксируйте:

Что получает система?

Например:

```
git diff
```

или:

```
error.log
```

или:

```
bug_report.md
```

Что система должна вернуть?

Например:

```
review.md
```

или структурированный ответ:

```
{
  "problem": "Возможная ошибка обработки None",
  "file": "service.py",
  "severity": "medium",
  "recommendation": "Добавить проверку значения перед вызовом метода"
}
```

Не передавайте ИИ лишние данные

Передавайте только тот контекст, который необходим для выполнения задачи.

Не отправляйте:

- токены;
- пароли;
- `.env`;
- ключи API;
- пользовательские персональные данные;
- production-секреты;
- конфиденциальную информацию компании без соответствующего разрешения.

Если используется рабочий проект, при необходимости подготовьте обезличенный или демонстрационный вариант.

Используйте ограниченные разрешения

Если агенту требуется доступ к инструментам, выдавайте только необходимые права.

Например, агенту для review обычно достаточно:

- чтения файлов;
- получения `git diff`;
- запуска тестов;
- запуска линтера.

Ему не обязательно давать:

- удаление репозитория;
 - работу с production;
 - доступ к секретам;
 - автоматический merge;
 - изменение инфраструктуры.
-

Проверяйте результат модели

Результат ИИ не должен автоматически считаться правильным.

Особенно проверяйте:

- изменения кода;
- команды;
- зависимости;
- тесты;
- SQL-запросы;
- конфигурацию;
- операции Git;
- изменения CI/CD.

Если агент изменяет код, после этого желательно выполнить обычные технические проверки проекта.

Не оценивайте решение только по одному хорошему ответу

LLM может дать правильный результат в одном запуске и ошибиться в следующем.

Поэтому желательно подготовить несколько контрольных примеров.

Например:

Тест 1 – обычная ошибка
Тест 2 – файл без ошибок
Тест 3 – неполные данные
Тест 4 – несколько ошибок одновременно
Тест 5 – неизвестная агенту ситуация

Что должно быть в итоговом проекте

Минимальный состав итоговой работы:

1. Репозиторий или отдельная директория проекта.
 2. Краткое описание рабочей задачи.
 3. Описание предлагаемого AI-решения.
-

4. Рабочая реализация или рабочий прототип.
5. Инструкции или prompts для ИИ.
6. Если используются агенты — описание их ролей.
7. Примеры входных данных.
8. Примеры результатов работы.
9. Описание способа проверки.
10. README с инструкцией запуска.
11. Описание ограничений решения.
12. Итоговое заключение о применимости решения в реальной работе.

Рекомендуемая структура репозитория

Структура может отличаться в зависимости от вашего проекта.

Пример:

```
project/  
  README.md  
  src/  
  tests/  
  prompts/  
  examples/  
  docs/  
  agents.md  
  requirements.txt
```

Не нужно создавать пустые директории только ради соответствия примеру.

Если проект уже существует, используйте его текущую структуру.

Что написать в README

README должен позволить другому человеку понять проект без дополнительных объяснений автора.

Укажите:

1. Название проекта

Например:

AI Reviewer для предварительной проверки Pull Request

2. Исходная рабочая задача

Что выполнялось до внедрения решения.

3. Цель

Какую часть процесса требуется автоматизировать или ускорить.

4. Используемые технологии

Например:

```
Python  
Git  
GitLab  
LLM API  
pytest
```

Указывайте только реально использованные технологии.

5. Архитектура решения

Кратко опишите компоненты и ответственность каждого из них.

6. Как запустить

Приведите реальные команды.

Например:

```
python -m venv .venv
```

```
pip install -r requirements.txt
```

```
python main.py
```

7. Как проверить

Например:

```
pytest
```

8. Пример использования

Покажите входные данные и полученный результат.

9. Ограничения

Опишите известные проблемы и условия использования.

10. Возможные дальнейшие улучшения

Укажите, что можно было бы сделать при дальнейшем развитии решения.

Что проверить перед отправкой: чек-лист

Перед сдачей итогового проекта проверьте:

- ☐ Выбрана реальная или максимально приближенная к реальной рабочая задача.
 - ☐ Понятно описано, какую проблему решает проект.
 - ☐ Определён ожидаемый результат.
 - ☐ Понятно, зачем в этой задаче используется ИИ.
 - ☐ Выбрано минимально достаточное решение.
 - ☐ Не добавлены лишние агенты только ради демонстрации multi-agent подхода.
 - ☐ Если используются несколько агентов, у каждого есть понятная отдельная роль.
 - ☐ Определено, какие действия выполняет ИИ, а какие остаются за человеком.
 - ☐ Есть инструкции или prompts, используемые системой.
 - ☐ Если используется `agents.md`, его содержимое соответствует реальным ролям агентов.
 - ☐ Подключены только действительно необходимые Tools.
 - ☐ В репозитории отсутствуют API-ключи, токены, пароли и другие секреты.
 - ☐ Решение работает хотя бы на основном сценарии.
 - ☐ Проверено несколько разных входных примеров.
 - ☐ Обработан хотя бы один ошибочный или пограничный сценарий.
 - ☐ Если проект работает с кодом, результат проверяется тестами, линтером или другим подходящим способом.
 - ☐ Используется Git.
 - ☐ В истории есть осмысленные коммиты.
 - ☐ Если CI подходит для этой задачи, показана или реализована соответствующая интеграция.
 - ☐ README содержит описание проекта.
 - ☐ README содержит инструкцию запуска.
 - ☐ README содержит пример использования.
 - ☐ Указаны ограничения AI-решения.
-

- [] Указано, где требуется ручная проверка.
- [] Есть описание результата «до применения ИИ» и «после применения ИИ».
- [] Можно продемонстрировать работу проекта преподавателю.

Советы по улучшению работы

Используйте измеримый критерий успеха

Вместо:

«ИИ ускоряет работу»

лучше указать:

«Ручная подготовка предварительного review занимала в среднем около 10 минут. При использовании прототипа первичный отчёт формируется автоматически, после чего разработчик выполняет ручную проверку».

Если точное измерение времени невозможно, используйте другие проверяемые показатели.

Сохраняйте результаты запусков

Для демонстрации полезно сохранить несколько примеров.

Например:

```
examples/  
  example_01_input.txt  
  example_01_result.md  
  
  example_02_input.txt  
  example_02_result.md  
  
  example_03_input.txt  
  example_03_result.md
```

Это позволяет показать не только код, но и фактическую работу решения.

Добавьте защиту от неправильных действий

Для агента можно явно установить ограничения.

Например:

```
Не изменяй файлы без подтверждения.  
Не выполняй git push.  
Не выполняй merge.  
Не обращай к production.  
Не используй данные из .env.  
Если информации недостаточно, сообщи об этом вместо генерации предположения.
```

Добавьте контроль качества результата

Можно реализовать дополнительную проверку:

- Reviewer проверяет результат Coder;
- тесты проверяют сгенерированный код;
- linter проверяет стиль;
- схема JSON проверяет структуру ответа;
- человек утверждает изменение перед записью файла.

Если проект большой — ограничьте контекст

Не передавайте агенту весь репозиторий без необходимости.

Можно использовать:

- поиск по имени;
- `grep`;
- `git diff`;
- список изменённых файлов;
- выбор конкретной директории;
- поиск релевантных документов;
- RAG.

Например:

```
git diff --name-only
```

позволяет сначала определить изменённые файлы, а затем работать только с ними.

Добавьте журналирование

Полезно сохранять:

- какой запрос получил агент;
- какие инструменты вызвал;
- какие файлы прочитал;
- какое решение предложил;
- где возникла ошибка.

Это особенно важно для агентных решений, где выполнение состоит из нескольких действий.

Отделяйте рекомендации ИИ от фактических результатов инструментов

Например:

```
Результат pytest:  
12 passed, 1 failed
```

является фактом.

А:

```
ИИ предполагает, что причина ошибки связана с неправильной обработкой null.
```

является гипотезой.

В итоговой работе желательно явно различать эти два типа информации.

Не скрывайте неудачные результаты

Если агент первоначально выполнял задачу неправильно, это можно включить в проект.

Например:

В первой версии prompt агент пытался анализировать весь репозиторий, из-за чего в контекст попадало слишком много нерелевантных файлов. После изменения процесса сначала выполняется получение списка изменённых файлов через Git, затем анализируется только соответствующий код.

Такой пример показывает реальную работу с AI-системой лучше, чем только один успешный запуск.

Что не требуется в итоговом проекте

Не требуется:

- создавать собственную LLM;
- обучать нейросеть;
- строить большую production-инфраструктуру;
- обязательно использовать RAG;
- обязательно использовать MCP;
- обязательно использовать несколько агентов;
- обязательно использовать конкретный агентный фреймворк;
- обязательно разворачивать полноценный CI/CD;
- обязательно интегрироваться с production;
- создавать сложный пользовательский интерфейс;
- создавать отдельный большой учебный проект, если можно использовать собственную рабочую задачу.

Основной принцип итоговой работы:

возьмите небольшую реальную задачу со своего текущего места работы, примените к ней подходящий AI-инструмент или AI-агента, получите работающий результат и покажите, как вы проверили его качество.

Итоговый проект должен демонстрировать не сложность системы, а осознанное применение подходов курса: постановку задачи, выбор архитектуры, управление агентом, работу с инструментами, контроль результата, безопасность и интеграцию решения в существующий процесс разработки.
